COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate course for games

03

Module 10: Signals, optimization and numerical foundations

Expanded Volume 02 / individual teaching module

Four lessons with derivations, runnable examples, practical work and 20 quiz questions.

PACKAGE 03 / EXPANDED VOLUME 02 / SEPTEMBER 28, 2026

Self-study: complete the derivations, implementation, testing and review.

Module 10 — Signals, optimization and numerical foundations

Original prerequisites: Modules 07-09.

Original module project: Complete Gate B: a mathematical engineering notebook plus tested transforms, numerical integration, signal filtering and an uncertainty-aware experiment.

Expanded project: Gate B: integrated game-mathematics notebook. The accumulated projects plus a sampling/filtering demonstration, a gradient check, and a convergent/nonconvergent optimization comparison, assembled into one reproducible handoff.

Acceptance: Satisfy CAPSTONE_SPEC.pdf: proofs, tested transforms, analytic dynamics/refinement, aliasing/filtering, probability checks, a measured experiment, data provenance, and an independent reproduction. These reuse module work; they are not five extra unrelated capstones.

This module has four lessons, 20 lesson quiz questions, 15 written problems and five review questions. The written problems and integration project are in the workbook.

Lesson 10.1 — Multivariable calculus and gradients

Learning outcome: Use partial derivatives to reason about spatial fields and optimization.

Change one input at a time

A scalar field f(x,y) maps a two-component input to one number, such as a cost or height. Its partial derivative with respect to x changes x while holding y fixed. The gradient collects these partial derivatives. For f=x^2+3y^2, grad(f)=(2x,6y).

The gradient points toward steepest local increase under the ordinary Euclidean metric. Its magnitude and direction depend on coordinate scaling and units. Replacing meters by millimeters changes numerical derivatives even when the underlying problem is equivalent. Normalize or scale parameters deliberately rather than treating arbitrary component magnitudes as physically comparable.

For a unit direction u, the directional derivative is grad(f)·u. Without a unit direction, that dot product includes the magnitude of the proposed input change. The first-order approximation is f(p+delta) approximately f(p)+grad(f)(p)·delta.

A Jacobian generalizes the derivative

For a vector-valued mapping F from n inputs to m outputs, the Jacobian is an m-by-n matrix with entry Jij=partial Fi/partial xj. It maps a small input displacement into an approximate output displacement. A gradient is associated with a scalar output; a Jacobian can represent an entire coordinate transformation or multi-output constraint.

For F(x,y)=(x^2+y,xy), J=[[2x,1],[y,x]]. At (2,3), J=[[4,1],[3,2]]. Applying it to delta=(0.01,-0.02) predicts delta_F=(0.02,-0.01). The true finite change also contains higher-order terms, so exact equality is not expected for a nonzero step.

Follow the chain rule through every transform

For a scalar cost g(F(x)), the gradient with respect to x is J_F(x)^T*grad(g)(F(x)) in this column-vector convention. The transpose appears because each input can influence several outputs. Omitting a transform or applying the Jacobian in the wrong direction can produce a vector of the correct length but the wrong meaning.

This is the mathematical foundation of gradient-based fitting and, later, reverse-mode differentiation. It is useful to derive a two-variable case manually before depending on an automatic-differentiation tool.

Stationary does not always mean minimum

A stationary point has zero gradient. For f=x^2+y^2, the origin is a minimum. For f=-(x^2+y^2), it is a maximum. For f=x^2-y^2, it is a saddle: some directions increase the value and others decrease it. The Hessian, a matrix of second partial derivatives, helps classify local curvature under suitable smoothness assumptions.

Constraints can also make a boundary point optimal even when its unconstrained gradient is nonzero. A zero-gradient stopping rule is appropriate only for the optimization problem actually being solved.

Check derivatives, but do not confuse a check with a proof

Centered differences perturb one coordinate at a time while holding others fixed. The reference uses the same h for each coordinate as a transparent educational choice. For strongly different scales, choose scale-aware perturbations and report them. Test several h values and include stationary points and nonzero points.

A smooth analytic gradient and a finite difference should agree within an error budget where truncation and roundoff are controlled. They can disagree because of a coding defect, a nonsmooth operation, a bad perturbation scale, or inconsistent coordinate conventions. A single matching point is weak evidence.

For a y-up terrain represented as y=h(x,z), a convenient unnormalized surface normal is (-partial h/partial x,1,-partial h/partial z). This explicitly matches the geometry convention rather than silently treating z as up. [R41, R07]

Worked derivation

For f(x,y)=x^2+3y^2 at (2,-1), the gradient is (4,-6) and the objective is 7. With step size alpha=0.1, a negative-gradient step gives (1.6,-0.4). The new objective is 1.6^2+3*(-0.4)^2=3.04.

This proves descent for the chosen step by direct evaluation; it does not prove that every step size works. At a sufficiently large step, the same correct gradient can increase the objective.

For the mapping F in the lesson, the exact change from (2,3) to (2.01,2.98) is (0.0201,-0.0102). The Jacobian prediction (0.02,-0.01) differs by second-order terms.

Runnable demonstration

Run `python run_example.py 10.1` from the extracted package folder. Predict before executing.

```
from mathkit import gradient
f = lambda p: p[0]**2 + 3*p[1]**2
point = (2.0, -1.0)
g = gradient(f, point, 0.0001)
print("numerical gradient:", tuple(round(x, 6) for x in g))
print("analytic gradient:", (2*point[0], 6*point[1]))
alpha = 0.1
new = tuple(x-alpha*dx for x, dx in zip(point, g))
print(f"objective: {f(point):.3f} -> {f(new):.3f}")
```

Recorded expected output:

```
numerical gradient: (4.0, -6.0)
analytic gradient: (4.0, -6.0)
objective: 7.000 -> 3.040
```

Practical assignment

Original brief: Derive gradients for three functions and check them with centered differences. Pass: test nonzero and stationary points, vary perturbation size and report errors rather than claiming exact equality.

1. Implement gradient(f,point,h), perturbing one coordinate at a time, rejecting invalid/unresolvable h and nonfinite evaluations.

2. Derive and check three gradients, including a coupled function such as x*y+x^2 and a function with a stationary saddle.
3. Derive the Jacobian example and compare predicted versus exact finite changes over several step magnitudes.
4. Create a y-up terrain-normal calculation and explicitly document which variables parameterize the surface.

Coding slice: Work in `labs/student/lab_10_1.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

A gradient is local and coordinate-dependent. A zero gradient can describe a saddle or maximum, and a correct gradient does not make every step size safe.

Independent transfer task

For the y-up height field h(x,z)=x^2+z^2, find a unit surface normal at (x,z)=(1,0).

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What is the gradient of x*x + 3*y*y?
2. What does a Jacobian describe?
3. Does a zero gradient always identify a minimum?
4. What is the Jacobian of F(x,y)=(x^2+y,xy)?
5. Classify the origin for f=x^2-y^2.

Assigned study

R41: partial derivatives, tangent approximations, chain rule, gradients, and directional derivatives. R07: linear maps and transposes. Complete the multivariable bridge before relying on this material in graduate optimization.

Original reading assignment retained: R07: linearization foundations; R23: geometric differential quantities; use the program's multivariable extension reading.

Lesson 10.2 — Sampling, aliasing and Fourier ideas

Learning outcome: Explain why discrete samples can misrepresent a continuous signal.

Samples do not uniquely determine every continuous signal

Sampling at rate fs observes a signal at t_n=n/fs. A sinusoid with frequency f has discrete phase increments 2*pi*f/fs. Adding an integer multiple of fs to its frequency adds an integer number of complete revolutions at every sample, leaving those sample values unchanged.

High-frequency content can therefore appear as a different lower-frequency signal. This is aliasing, not random numerical noise. A baseband alias magnitude can be found by reducing the frequency modulo fs and reflecting values above fs/2. That magnitude alone does not preserve phase or the sign relationship between sampled sines.

For cosine, frequencies 900 Hz and 100 Hz sampled at 1000 Hz give the same samples because cos(2*pi*0.9n)=cos(2*pi*n-2*pi*0.1n)=cos(2*pi*0.1n). For sine the corresponding relation has a minus sign. Saying merely "900 becomes positive 100" would lose that phase information.

Bandwidth assumptions make reconstruction possible

An ideal sampling theorem applies to appropriately band-limited signals sampled above twice their highest frequency. The half-rate fs/2 is the Nyquist boundary. A sinusoid exactly at that boundary can lose phase information; for example, sin(pi*n)=0 for every integer n. Practical filters need transition bands and margin, not only an equality in a specification.

Increasing sample rate helps relative to the signal bandwidth. It does not justify sampling arbitrary sharp edges without filtering. Similarly, a high-resolution texture can still alias when it is reduced to only a few screen pixels.

Filter before discarding samples

Downsampling keeps fewer samples, lowering the representable bandwidth. A low-pass filter applied first suppresses components that would otherwise fold into misleading lower frequencies. Filtering after aliasing has occurred cannot generally identify which original signal produced the samples.

The checker sequence [0,1,0,1,...] illustrates this. Keeping every other value can produce all zeros or all ones depending on phase. Averaging adjacent pairs before keeping one value yields 0.5 for this exact pattern. The two-tap average solves that chosen example; it is not an ideal anti-alias filter for every possible signal.

Fourier components provide another view

Complex numbers introduce i with i^2=-1. Euler's identity exp(i*theta)=cos(theta)+i*sin(theta) packages phase conveniently. A discrete Fourier transform of N samples is X[k]=sum_n x[n]*exp(-i*2*pi*k*n/N). Under this convention the inverse has a 1/N factor and a positive exponent.

Bin k corresponds to k*fs/N before accounting for the wrapped negative-frequency interpretation of upper bins. Real signals have conjugate symmetry. A finite observation window can spread a sinusoid's energy across bins when it does not align with an integer number of cycles; this is spectral leakage, distinct from aliasing.

For x=[1,0,-1,0], the unnormalized four-point DFT is [0,2,0,2]. The two nonzero bins describe the positive and negative parts of a real cosine. A faster FFT computes the same transform family; it is not a different physical model.

Keep time and space analogous but explicit

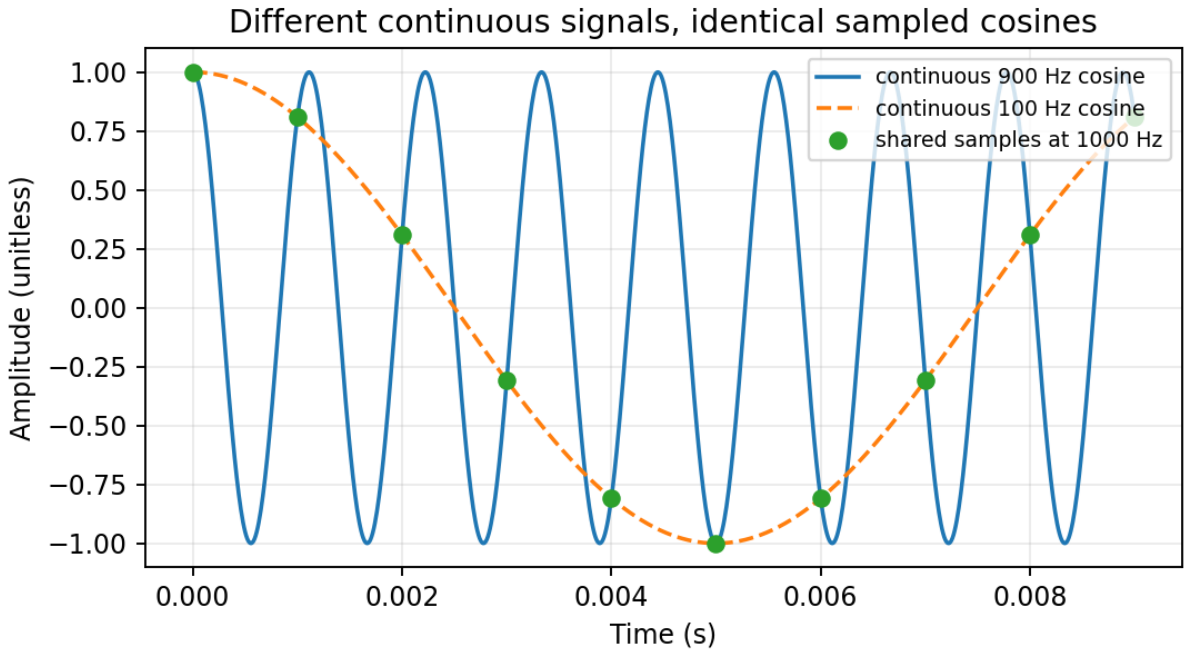
Audio samples are spaced in time, texture samples in space, and animations may be sampled in both. The same sampling reasoning applies, but their units and reconstruction assumptions differ. Record sample rate or spacing, frequency units, filter, boundary policy, and decimation phase. [R10, R23]

Worked derivation

At fs=1000 Hz, the samples of cos(2*pi*900*t) match cos(2*pi*100*t). The demonstration evaluates ten sample indices and checks that their maximum numerical disagreement is below 1e-12 in the tested environment. The trigonometric identity, not those ten checks alone, explains all integer sample indices in exact arithmetic.

For checker=[0,1,0,1,0,1,0,1], raw decimation starting at index zero gives [0,0,0,0]. Adjacent pair averages give [0.5,0.5,0.5,0.5]. Starting the raw decimation at index one would instead yield all ones.

At 48,000 samples/second, the Nyquist boundary is 24,000 Hz. It is a boundary in an idealized analysis, not a guarantee that a practical filter passes everything up to it perfectly.



Continuous cosines differ between samples, but the marked 1000 Hz samples coincide for the chosen 900 Hz and 100 Hz cosines.

Runnable demonstration

Run `python run_example.py 10.2` from the extracted package folder. Predict before executing.

```
import math
from mathkit import alias_frequency
print("alias magnitude:", alias_frequency(900, 1000), "Hz")
# Cosines match; a 900 Hz sine would be the NEGATIVE of the 100 Hz sine.
errors = [abs(math.cos(2*math.pi*900*n/1000) - math.cos(2*math.pi*100*n/1000)) for n in range(10)]
print("cosine samples agree within 1e-12:", max(errors) < 1e-12)
checker = [0, 1]*4
print("raw decimation:", checker[::2])
print("pair-averaged decimation:", [(checker[i]+checker[i+1])/2 for i in range(0, len(checker),
    2)])
```

Recorded expected output:

```
alias magnitude: 100.0 Hz
cosine samples agree within 1e-12: True
raw decimation: [0, 0, 0, 0]
pair-averaged decimation: [0.5, 0.5, 0.5, 0.5]
```

Practical assignment

Original brief: Generate low- and high-frequency sine samples, then downsample a checker pattern with and without prefiltering. Pass: explain the artifacts using frequency content rather than calling them random noise.

1. Implement alias_frequency(frequency,sample_rate), returning only the nonnegative baseband magnitude and documenting that phase is not encoded.
2. Generate the 900/100 Hz cosine pair and the corresponding signed sine relation at fs=1000. Verify both analytically and numerically.
3. Downsample a checker pattern with both starting phases and with a declared prefilter. Do not hide the filter delay or boundary behavior.
4. Compute the four-point DFT example by hand and explain why this is a different issue from the alias demonstration.

Coding slice: Work in `labs/student/lab_10_2.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Do not confuse aliasing with random noise or spectral leakage, omit the sine phase inversion, or claim a two-tap average is a universal reconstruction filter.

Independent transfer task

A cosine at 700 Hz is sampled at 1000 Hz. Find its baseband alias magnitude and explain a sine caveat.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What is the Nyquist frequency at 48,000 samples/second?
2. Why filter before downsampling?

3. Can post-filtering always undo aliasing?
4. Why is sampling exactly at twice a sinusoid frequency risky for phase information?
5. What distinguishes spectral leakage from aliasing?

Assigned study

R10: sampling and Fourier representations. R23: sampling and reconstruction. The complex-number and DFT conventions here are an explicit Package 03 bridge, not an additional numbered module.

Original reading assignment retained: R10: sampling and Fourier representation; R23: Sampling and Reconstruction.

Lesson 10.3 — Convolution, filters and systems

Learning outcome: Predict a filter's effect on data and latency.

A filter is a mathematical operation with boundaries

For discrete signal x and kernel h, convolution is y[n]=sum_k h[k]*x[n-k]. The n-k indexing shifts and reverses the kernel relative to the input. A routine using x[n+k] without a corresponding reindexing generally computes a correlation-like operation instead.

This bundle defines finite full convolution with zero extension outside the input. If the signal has N values and the nonempty kernel has K values, the output has N+K-1 values. An empty signal produces an empty output. Other libraries may use "same" or "valid" modes with different alignment; those names do not remove the need to specify the contract.

Derive a system from its impulse response

A unit impulse is one at index zero and zero elsewhere. A discrete signal can be written as a sum of shifted impulses weighted by its samples. For a linear time-invariant system, each shifted impulse produces a correspondingly shifted response, and superposition adds the weighted responses. That yields convolution with the impulse response.

Linearity means preserving weighted sums. Time invariance means an input shift produces the same output shift. A clamp is nonlinear; a filter whose coefficients change each frame is generally not time-invariant. Do not assume the impulse-response model describes every game subsystem.

Smoothing changes delay and detail

A three-sample moving average has coefficients (1/3,1/3,1/3). In a causal implementation it uses the current and two previous samples. It suppresses some rapid variation but also rounds edges and delays features. A symmetric length-K causal finite-impulse-response filter has linear phase with group delay (K-1)/2 samples where its frequency response is nonzero.

For K=3 at fs=1000 samples/second, that delay is one sample, or 1 ms. It is only the filter's contribution, not total controller-to-display latency. A centered offline filter may align the result differently, but it uses future samples unavailable to a real-time causal system.

The sum of coefficients explains constant signals

For a constant infinite signal c, convolution gives c*sum(h). A unity-sum kernel therefore preserves the constant in the interior or steady-state region. At finite boundaries with zero extension, the kernel overlaps missing samples and the output can ramp or fall. That is expected under the chosen boundary policy.

Zero padding, edge replication, reflection, and wrapping represent different assumptions about data outside the recorded region. A wrapped terrain texture may need a different policy from an audio segment that starts in silence. Make the choice explicit and test it.

Numeric and implementation checks

Test an impulse to reveal the kernel and its alignment. Test a constant to reveal gain and boundary effects. Test a step to reveal smoothing and delay. Use an asymmetric kernel as well as a symmetric one, because reversing a symmetric kernel can hide a convolution-versus-correlation error.

Every input and coefficient in the exercise must be finite. Large coefficients can amplify noise and overflow an implementation even when the formula is well defined in real arithmetic. A short direct convolution has cost proportional to N*K; FFT-based alternatives have different overhead and padding requirements, and are not automatically better for a tiny kernel.

When filtering colors, the numerical values must be interpreted in the intended space; averaging encoded display values is not automatically the same operation as averaging linear light. That detailed rendering issue returns in Volume 5. Here the scalar signal's meaning and units are declared directly. [R10, R23]

Worked derivation

For signal [1,2,3] and kernel [0.5,0.5], zero-extended full convolution is [0.5,1.5,2.5,1.5]. At n=1, the calculation is 0.5*x[1]+0.5*x[0]=1.5. At the last output, only the final input overlaps the kernel.

For signal [1], the output equals the kernel exactly, showing the impulse response. For a constant four-sample input [1,1,1,1], the same kernel produces [0.5,1,1,1,0.5]. Taking the first four values gives a causal same-length prefix; the initial 0.5 is the declared zero-history transient.

With asymmetric kernel [1,2] and signal [1,2,3], full convolution gives [1,4,7,6]. Keep this as a test that a symmetric moving average cannot replace.

Runnable demonstration

Run `python run_example.py 10.3` from the extracted package folder. Predict before executing.

```
from mathkit import convolve
print("full convolution:", convolve([1, 2, 3], [0.5, 0.5]))
print("impulse response:", convolve([1], [0.25, 0.5, 0.25]))
x = [1, 1, 1, 1]
print("causal same-length prefix:", convolve(x, [0.5, 0.5])[:len(x)])
```

Recorded expected output:

```
full convolution: [0.5, 1.5, 2.5, 1.5]
impulse response: [0.25, 0.5, 0.25]
causal same-length prefix: [0.5, 1.0, 1.0, 1.0]
```

Practical assignment

Original brief: Implement a one-dimensional convolution with documented boundary handling. Test an impulse, a constant and a step. Pass: predict the output before running and identify the filter's delay convention.

1. Implement convolve(signal,kernel) as full zero-extended convolution. Return [] for empty signal, reject empty kernel and nonfinite data.
2. Predict and test impulse, constant, step, and asymmetric-kernel outputs. Include output length and alignment checks.

3. Compare full output with a causal same-length prefix and with a separately specified boundary alternative.
4. Explain gain from sum(kernel), the filter delay in samples and seconds, and why a future-looking centered filter is different in real time.

Coding slice: Work in `labs/student/lab_10_3.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Boundary handling and output alignment change the result. Unity-sum coefficients preserve a constant in steady state, not necessarily at zero-padded finite edges.

Independent transfer task

A symmetric five-tap causal filter runs at 1000 Hz. What is its linear-phase group delay, and what does it exclude?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What does convolving with an impulse reveal?
2. What is the average of 2, 5 and 8?
3. Why is boundary handling not an implementation footnote?
4. What is the full convolution output length for N=4 and K=3, under this bundle contract?
5. Why test an asymmetric kernel?

Assigned study

R10: convolution, impulse and step responses, linear time invariance, and frequency response. R23: later image-filtering context.

Original reading assignment retained: R10: convolution, impulse response and linear time-invariant systems.

Lesson 10.4 — Optimization, conditioning and approximation budgets

Learning outcome: Choose an objective and distinguish convergence from correctness.

Choose what is being optimized

An optimization problem has variables, an objective, and constraints. "Make the game better" is not a mathematical objective. Minimizing a fitting error, minimizing time while keeping error below a threshold, and minimizing memory under a latency budget are different problems.

A local minimum need not be global. Convex objectives over convex feasible regions have useful global properties, but those assumptions must be established rather than inferred from a smooth-looking graph. This lesson begins with a quadratic whose behavior can be derived exactly.

Derive fixed-step gradient descent

For f(x)=(x-3)^2, the derivative is 2(x-3). A gradient step is x_new=x-alpha*2(x-3). Define error e=x-3; then e_new=(1-2alpha)*e. Repeating gives e_n=(1-2alpha)^n*e_0.

The error decays exactly when |1-2alpha|<1, giving 0<alpha<1. At alpha=0.5 the exact-arithmetic update reaches the minimizer in one step. At alpha=1 it alternates with unchanged error magnitude; above one it diverges except from the exact minimizer. A correct gradient is compatible with a bad step-size choice.

For f(x,y)=(x-3)^2+3(y+1)^2, the two error factors are 1-2alpha and 1-6alpha. Both must have magnitude below one, so 0<alpha<1/3. The steeper direction imposes the tighter condition. This connects curvature, scaling, and the eigenvalues studied earlier.

Stop with a declared status

A useful solver reports its achieved point, objective, gradient or residual, iteration count, and reason for stopping. Reaching an iteration limit is not convergence. A nonfinite result is not an acceptable approximate answer and should trigger an explicit failure path.

A small step norm is also not enough by itself: an extremely small alpha can make progress tiny far from the solution. For this particular one-dimensional quadratic, |f'(x)|<=tol implies |x-3|<=tol/2. That bound follows from the known derivative and does not automatically apply to arbitrary objectives.

The reference uses gradient tolerance and an iteration cap. It retains history for inspection and raises an arithmetic error on a nonfinite trajectory rather than emitting a false success record.

Constraints can change the optimality condition

With the added constraint x>=4, the minimizer of (x-3)^2 is x=4, where the unconstrained gradient is 2, not zero. Projecting a proposed point back into an interval is one possible algorithmic building block, but it changes the convergence analysis. Detailed constrained optimality and multipliers are reserved for the later graduate sequence.

In engineering, a quality limit is often a hard constraint. Adding a small penalty for violating it does not guarantee the limit will be respected. Check feasibility independently of the objective value.

Budget approximation error rather than hiding it

An application can accumulate model, discretization, sampling, roundoff, and iterative-solver errors. If defensible worst-case absolute bounds apply to quantities in the same output units, the triangle inequality permits adding them conservatively. Root-sum-of-squares aggregation needs additional stochastic assumptions; it is not a universal shortcut.

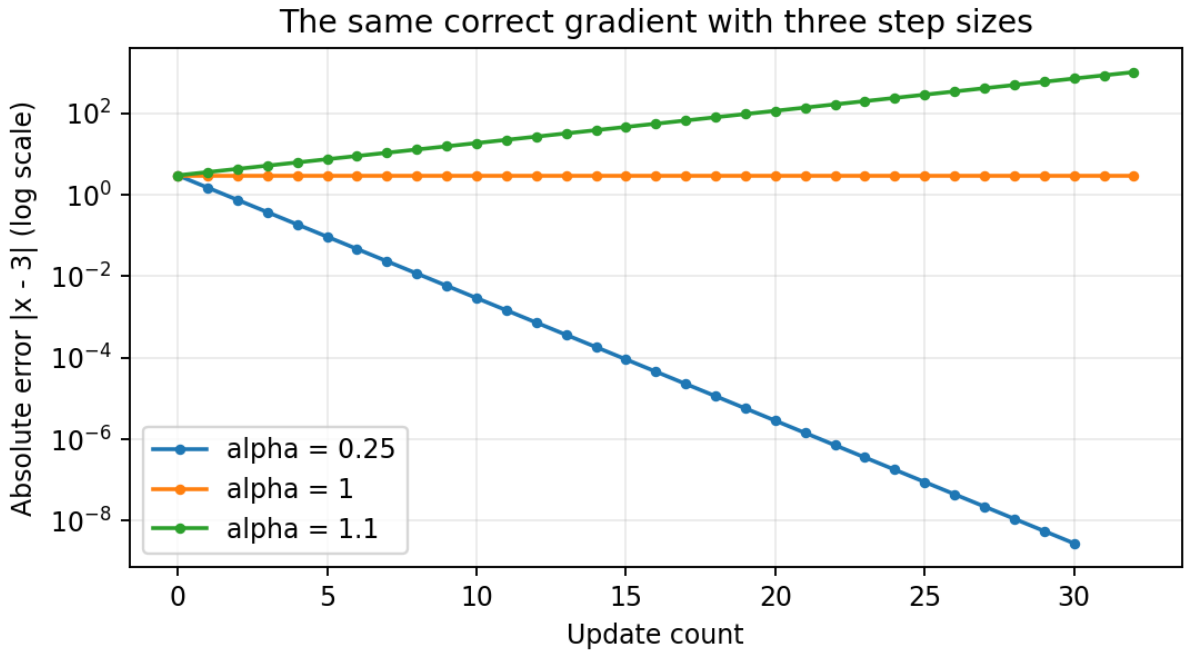
Identify a reference case, state which errors are controlled, and record the remaining uncertainty. More solver iterations reduce only the error the solver is responsible for. They cannot fix an incorrect model or aliased input. Completing Gate B means defending those distinctions with equations, tests, and reproducible data. [R42, R07; quadratic derivations in this lesson]

Worked derivation

Start at x0=0, so e0=-3. At alpha=0.25, the error factor is 0.5. After eight updates, x=3-3/256=2.98828125 and |gradient|=0.0234375. A tolerance of 0.03 is met.

At alpha=1, the error factor is -1, so the iterates alternate 0,6,0,6,... and do not converge from this start. At alpha=1.1, the factor is -1.2; after eight updates x=3-3*(1.2)^8, approximately -9.899451. The reference reports the iteration limit, not success.

Suppose known absolute output-error bounds are 0.01,0.02,0.005 in the same units. A conservative combined bound is 0.035. Calling sqrt(0.01^2+0.02^2+0.005^2) a guaranteed bound would require assumptions not supplied here.



Absolute scalar-quadratic error for selected step sizes. The alpha=1 trace oscillates in signed error while its magnitude stays fixed.

Runnable demonstration

Run `python run_example.py 10.4` from the extracted package folder. Predict before executing.

```
from mathkit import descent_quadratic
for alpha in (0.25, 1.0, 1.1):
    result = descent_quadratic(0, alpha, max_steps=8, tol=0.03)
    print(f"alpha={alpha:.2f} x={result['x']:.6f} steps={result['iterations']}
    reason={result['reason']}")
```

Recorded expected output:

```
alpha=0.25 x=2.988281 steps=8 reason=gradient_tolerance
alpha=1.00 x=0.000000 steps=8 reason=iteration_limit
alpha=1.10 x=-9.899451 steps=8 reason=iteration_limit
```

Practical assignment

Original brief: Implement gradient descent on a quadratic and a two-variable problem. Pass: compare step sizes, record residuals and state a stopping rule; complete Gate B with no unsupported claim of global optimality.

1. Implement descent_quadratic(x0,alpha,max_steps,tol) with explicit convergence/iteration-limit status, achieved gradient, and a history of iterates.
2. Compare alpha=0.25,0.5,1,1.1 from the same start. Preserve nonconvergent cases and verify reported status at the final allowed iteration.
3. Implement a two-variable extension and derive its step-size restriction from the two error factors, rather than guessing by trial alone.
4. Complete Gate B: assemble proofs, tested geometry, dynamics/refinement, sampling/filtering, probability, measured experiment, and a reproducibility handoff.

Coding slice: Work in `labs/student/lab_10_4.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Reaching the iteration cap or taking tiny steps does not mean convergence. A constrained optimum need not have zero unconstrained gradient, and error terms cannot be combined with unjustified independence assumptions.

Independent transfer task

For f(x,y)=(x-3)^2+3(y+1)^2, compare alpha=0.1 and alpha=0.4.

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Where is the minimum of (x-3)^2?
2. Why can reducing step size help an unstable iteration?
3. What should happen when an iterative method reaches its iteration limit?
4. What range of positive alpha makes the scalar quadratic iteration converge from every finite start in exact arithmetic?

5. Why can x=4 be a constrained optimum even though f prime(4)=2 for f=(x-3)^2?

Assigned study

R42: objectives, convexity, and descent concepts as a bridge; the full graduate optimization treatment comes later. R07: conditioning and least squares. Use the explicit error-factor derivation for this lab.

Original reading assignment retained: R07: least squares and conditioning; R23: numerical utilities; graduate optimization reading in the handbook.